

THREE-PRECISION ITERATIVE REFINEMENT WITH PARAMETER REGULARIZATION AND PREDICTION FOR SOLVING LARGE SPARSE LINEAR SYSTEMS*

JIFENG GE[†], JUAN ZHANG[†], AND JIANZHOU LIU[†]

Abstract. This study presents a novel mixed-precision iterative refinement algorithm, GADI-IR, within the general alternating-direction implicit (GADI) framework, designed for efficiently solving large-scale sparse linear systems. By employing low-precision arithmetic, particularly half-precision (FP16), for computationally intensive inner iterations, the method achieves substantial acceleration while maintaining high numerical accuracy. Key challenges such as overflow in half-precision and convergence issues for low precision are addressed through careful backward error analysis and the application of a regularization parameter α . Furthermore, the integration of low-precision arithmetic into the parameter prediction process, using Gaussian Process Regression (GPR), significantly reduces computational time without degrading performance. The method is particularly effective for large-scale linear systems arising from discretized partial differential equations and other high-dimensional problems, where both accuracy and efficiency are critical. Numerical experiments demonstrate that the use of mixed-precision strategies not only accelerates computation but also ensures robust convergence, making the approach advantageous for various applications. The results highlight the potential of leveraging lower-precision arithmetic to achieve superior computational efficiency in high-performance computing.

Key words. Mixed precision, iterative refinement, general alternating-direction implicit framework, large sparse linear systems, convergent analysis.

MSC codes. 65G50, 65F10

1. Introduction.

1.1. Background. Mixed precision techniques have been a focus of research for many years. With advancements in hardware, such as the introduction of tensor cores in modern GPUs, half-precision arithmetic (FP16) has become significantly faster than single or double precision [12][13], driving its growing importance in high-performance computing (HPC) and deep learning. By strategically utilizing FP16 alongside the capabilities of modern tensor cores, mixed precision computing marks a major breakthrough in HPC. It offers notable improvements in computational speed, memory utilization, and energy efficiency, all while preserving the accuracy required for a wide array of scientific and engineering applications.

Solving large sparse linear systems is a cornerstone of numerical computing, with broad applications in scientific simulations, engineering problems, and data science. These systems frequently arise from the discretization of partial differential equations or the modeling of complex processes, where efficient and scalable solution methods are essential. In this context, the paper [10] introduces an innovative and flexible framework called the general alternating-direction implicit (GADI) method. This framework tackles the challenges associated with large-scale sparse linear systems by unifying existing methods under a general structure and integrating advanced strategies, such as Gaussian process regression (GPR)[16], to predict optimal parameters.

*Submitted to the editors DATE.

Funding: National Key Research and Development Program of China (Grant No. 2023YFB3001604).

[†]Key Laboratory of Intelligent Computing and Information Processing of Ministry of Education, Hunan Key Laboratory for Computation and Simulation in Science and Engineering, School of Mathematics and Computational Science, Xiangtan University, Xiangtan, Hunan, China, 411105. (Corresponding authors. zhangjuan@xtu.edu.cn(JZ)).

These innovations significantly enhance the computational efficiency, scalability, and robustness of solving such systems.

The motivation for incorporating mixed precision into the GADI framework stems from the growing need to balance computational efficiency and resource utilization when solving large-scale sparse linear systems. Mixed precision methods, which combine high precision (e.g., double precision) and low precision (e.g., single or half precision) arithmetic, have gained traction due to advancements in modern hardware, such as GPUs and specialized accelerators, which are optimized for lower-precision computations.

The use of low precision (like 16-bit floating-point) in the GADI framework offers a powerful combination of speed, memory efficiency, and energy savings, making it particularly well-suited for solving large sparse linear systems. FP16 computations are significantly faster than FP32 or FP64 on modern hardware like GPUs with Tensor Cores, enabling rapid execution of matrix operations while dramatically reducing memory usage, which allows larger problems to fit within the same hardware constraints. This reduced precision also lowers energy consumption, making it a more sustainable option for large-scale computations. While FP16 has limitations in range and precision, its integration into a mixed precision approach within the GADI framework ensures critical calculations retain higher precision to guarantee robustness and convergence. Additionally, FP16 accelerates the Gaussian Process Regression (GPR) used for parameter prediction, enabling efficient optimization of the framework's performance. Together, these advantages position FP16 as a key enabler for scalable and efficient numerical computing in modern applications.

Iterative refinement is a numerical technique used to improve the accuracy of a computed solution to a linear system of equations, particularly when the initial solution is obtained using approximate methods. It is widely used in scenarios where high precision is required, such as solving large-scale or ill-conditioned linear systems.

The process starts with an approximate solution, often computed in low precision for efficiency, followed by a series of iterative corrections. Each iteration involves:

- Residual Computation: Calculate the residual $r = Ax - b$, in precision u_f , where A is the coefficient matrix, x is the current solution.
- Solving a linear system to obtain a correction vector that reduces the residual in precision u_r .
- Updating the solution with the correction vector in precision u .

This process repeats until the residual or the correction falls below a specified tolerance, indicating convergence to the desired accuracy. The error analyses were given for fixed point arithmetic by [11] and [15] for floating point arithmetic.

Half precision(16-bit) floating point arithmetic, defined as a storage format in the 2008 revision of the IEEE standard [8], is now starting to become available in hardware, for example, in the NVIDIA A100 and H100 GPUs [12][13], on which it runs up to 50x as fast as double precision arithmetic with a proportional saving in energy consumption. Moreover, IEEE introduced the FP8 (Floating Point 8-bit) format as part of the IEEE 754-2018 standard revision which is designed with a reduced bit-width compared to standard IEEE floating-point formats like FP32 (32-bit floating point) and FP64 (64-bit floating point). So in the 2010s iterative refinement attracted renewed interest. The following table summarizes key parameters for IEEE arithmetic precisions.

Many famous algorithms for linear systems like gmres, LSE and etc. have been studied in mixed precision[6][7][2]. In this paper, we focus on the iterative refinement of the GADI framework[10].

Type	Size	Range	Unit roundoff u
half	16bits	$10^{\pm 5}$	$2^{-11} \approx 4.9 \times 10^{-4}$
single	32bits	$10^{\pm 38}$	$2^{-24} \approx 6.0 \times 10^{-8}$
double	64bits	$10^{\pm 308}$	$2^{-53} \approx 1.1 \times 10^{-16}$
quadruple	128bits	$10^{\pm 4932}$	$2^{-113} \approx 9.6 \times 10^{-35}$

Table 1: *Parameters for four IEEE arithmetic precisions. "Range" denotes the order of magnitude of the largest and smallest positive normalized floating point numbers.*

1.2. Challenges. Applying mixed precision to the GADI framework presents several challenges. The reduced numerical range and precision of formats like FP16 can lead to instability or loss of accuracy in iterative computations, particularly when solving ill-conditioned systems or handling sensitive numerical operations.

FP16 has a limited numerical range, with maximum and minimum representable values significantly smaller than those in FP32 or FP64. When solving large sparse linear systems, certain operations—such as scaling large matrices or intermediate results in iterative steps—may exceed this range, causing overflow. This can lead to incorrect results or instability in the algorithm, particularly when the system matrices have large eigenvalues or poorly conditioned properties. Careful rescaling techniques or mixed-precision strategies are often required to mitigate this issue, ensuring critical computations remain within the numerical limits of FP16.

The reduced precision of FP16 (approximately 3 decimal digits) can introduce rounding errors during iterative processes in GADI. These errors accumulate and may prevent the solver from reaching a sufficiently accurate solution, especially for problems that require high precision or have small residual tolerances. The convergence criteria may need to be relaxed when using FP16, or critical steps (e.g., residual corrections or parameter updates) must be performed in higher precision (FP32 or FP64) to ensure the algorithm converges to an acceptable solution.

In the GADI framework, the Gaussian Process Regression (GPR) method is used to predict the optimal parameter α to enhance computational efficiency. However, when applying mixed precision, particularly with FP16, the predicted α may fail to ensure convergence. This is because α is derived assuming higher precision arithmetic, and the reduced precision and numerical range of FP16 can amplify rounding errors, introduce instability, or exacerbate sensitivity to parameter choices. Consequently, the predicted α may no longer balance the splitting matrices effectively, leading to slower convergence or divergence in the mixed precision GADI framework. Addressing this issue may require recalibrating α specifically for mixed precision or employing adaptive strategies that dynamically adjust parameters during computation to account for FP16 limitations.

1.3. Contribution. To resolve the challenges of mixed-precision GADI, we proposed rigorous theoretical analysis and extensive empirical validation to ensure that the mixed-precision GADI with iterative refinement achieves its goals of accelerated computation and high numerical accuracy. In this article, we introduce a mixed precision iterative refinement method of GADI as GADI-IR to solve large sparse linear systems of the form:

$$(1.1) \quad Ax = b.$$

The goal of this work is to develop a mixed-precision iterative refinement method

that based on the GADI framework using three precisions. Our contributions are summarized as follows:

- We propose a novel mixed-precision iterative refinement method, GADI-IR, within the GADI framework, designed for efficiently solving large-scale sparse linear systems. We provide a rigorous theoretical analysis to ensure the numerical accuracy of the method.
- We address the challenges of overflow and underflow in FP16 arithmetic by applying a regularization parameter α to balance the splitting matrices effectively and ensure robust convergence in the inner low precision steps.
- We integrate low-precision arithmetic into the parameter prediction process using Gaussian Process Regression (GPR) method and compare the performance of GADI-IR with and without the regularization parameter. This demonstrates the effectiveness of α in enhancing computational efficiency and robustness.
- By applying GADI-IR to
 - Three-dimensional convection-diffusion equation,
 - Continuous-time algebraic Riccati equation (CARE),
 - Continuous Sylvester equation,

we demonstrate the effectiveness of the proposed method in solving large-scale sparse linear systems. The numerical experiments confirm that the mixed-precision approach significantly accelerates computation while maintaining high numerical accuracy. The results also highlight the importance of the regularization parameter α in ensuring robust convergence, particularly when using low-precision arithmetic.

1.4. Preliminaries. We now summarize our notation and our assumptions in this article.

For a non singular matrix A and a vector x , we need the normwise condition number:

$$(1.2) \quad \kappa(A) = \|A\| \cdot \|A^{-1}\|.$$

If the norm is the 2-norm, we denote the condition number as $\kappa_2(A)$:

$$(1.3) \quad \kappa_2(A) = \|A\|_2 \cdot \|A^{-1}\|_2 = \sigma_{\max}(A)/\sigma_{\min}(A).$$

$fl_r(\cdot)$ denotes the evaluation of the argument of fl_r in precision u_r .

The exact solution of $Ax = b$ is denoted by x and the computed solution is denoted by $\hat{x}$.

In algorithm GADI-IR, we use the following notation:

$$\begin{aligned} H &= \alpha I + M, \\ S &= \alpha I + N, \\ (2 - \omega)\alpha &= p. \end{aligned}$$

2. Error analysis.

2.1. GADI-IR framework. In [10], Jiang et al. proposed the GADI framework and corresponding algorithm.

Let $M, N \in \mathbb{C}^{n \times n}$ be splitting matrices of A such that: $A = M + N$. Given an

initial guess $x^{(0)}$, and $\alpha > 0, \omega > 0$, the GADI framework is:

$$(2.1) \quad \begin{cases} (\alpha I + M)x^{(k+\frac{1}{2})} = (\alpha I - N)x^{(k)} + b, \\ (\alpha I + N)x^{(k+1)} = (N - (1 - \omega)\alpha I)x^{(k)} + (2 - \omega)\alpha x^{(k+\frac{1}{2})}. \end{cases}$$

The following theorem, restated from [10], describes the convergence of the GADI framework:

THEOREM 2.1 (Convergence of the GADI Framework, Jiang et al., 2022). *The GADI framework (2.1) converges to the unique solution x of the linear system $Ax = b$ for any $\alpha > 0$ and $\omega \in [0, 2)$. Furthermore, the spectral radius $\rho(T(\alpha, \omega))$ satisfies:*

$$\rho(T(\alpha, \omega)) < 1,$$

where the iterative matrix $T(\alpha, \omega)$ is defined as:

$$T(\alpha, \omega) = (\alpha I + N)^{-1}(\alpha I + M)^{-1}(\alpha^2 I + MN - (1 - \omega)\alpha A).$$

The GADI framework for solving large sparse linear systems, including its full-precision error analysis and convergence properties have been thoroughly investigated[10]. It[10] unifies existing ADI methods and introduces new schemes, while addressing the critical issue of parameter selection by employing Gaussian Process Regression (GPR) method for efficient prediction. Numerical results demonstrate that the GADI framework significantly improves computational performance and scalability, solving much larger systems than traditional methods while maintaining accuracy.

Building on this foundation[10], we designed a mixed-precision GADI algorithm GADI-IR that strategically combines low-precision and high-precision computations. Computationally intensive yet numerically stable operations, such as matrix-vector multiplications, are performed in FP16 to leverage its speed and efficiency, while critical steps like residual corrections, parameter updates, and convergence checks are handled in higher precision to ensure robustness.

Algorithm 2.1 GADI-IR

Require: : $\alpha, \omega, \xi, H, S, \varepsilon, \hat{r}^{(0)}, k = 0, \hat{x}^{(0)} = 1$

while $\|\hat{r}^{(k)}\|_2^2 > \|\hat{r}^{(0)}\|_2^2 \xi$ **do**

Step 1: $\hat{r}^{(k)} = b - A\hat{x}^{(k)}$

$\triangleright u_f$

Step 2: Solve $H\hat{z}^{(k)} \approx \hat{r}^{(k)}$ such that $tol \leq \varepsilon\|\hat{r}^{(0)}\|_2$

$\triangleright u_r$

Step 3: Solve $S\hat{y}^{(k)} \approx (2 - \omega)\alpha\hat{z}^{(k)}$ such that $tol \leq \varepsilon\|\hat{r}^{(0)}\|_2$

$\triangleright u_r$

Step 4: Compute $\hat{x}^{(k+1)} = \hat{x}^{(k)} + \hat{y}^{(k)}$

$\triangleright u$

Step 5: Compute $k = k + 1$

We present the rounding error analysis of Algorithm 2.1 in the following sections, which include forward error bounds and backward error bounds in section 2. The significance of regularization to avoid underflow and overflow when half precision is used is explained in section 3. In this section we also specialize the results of Gaussian Process Regression (GPR) prediction to the GADI-IR algorithm and compare it with the regularization method. Numerical experiments presented in section 4 confirm the predictions of the analysis. Conclusions are given in section 5.

2.2. Forward analysis. Let step 2 and step 3 in Algorithm 2.1 performed using a backward stable algorithm, then there exists G_k and F_k such that:

$$(2.2) \quad (H + G_k)\hat{z}^{(k)} = \hat{r}^{(k)},$$

$$(2.3) \quad (S + F_k)\hat{y}^{(k)} = (2 - \omega)\alpha\hat{z}^{(k)},$$

where:

$$\|G_k\| \leq \phi(n)u_r\|M\|,$$

$$\|F_k\| \leq \varphi(n)u_r\|N\|.$$

where $\phi(n)$ and $\varphi(n)$ are reasonably small functions of matrix size n .

Considering the computation of $\hat{r}^{(k)}$, There are two stages. First, $\hat{s}^{(k)} = fl_f(b - A\hat{x}^{(k)})$ is formed in precision u_f , so that:

$$(2.4) \quad \|\hat{s}^{(k)}\| \leq \varphi_1(n)u_f(\|A\|\|\hat{x}^{(k)}\| + \|b\|).$$

Second, the residual is rounded to precision u_r , so $\hat{r}^{(k)} = fl_r(\hat{s}^{(k)}) = \hat{s}^{(k)} + f_k$. Hence:

$$(2.5) \quad \hat{r}^{(k)} = b - A\hat{x}^{(k)} + \Delta\hat{r}^{(k)},$$

where:

$$\|\Delta\hat{r}^{(k)}\| \leq u_r\|b - A\hat{x}^{(k)}\| + (1 + u_r)u_f(\|A\|\|\hat{x}^{(k)}\| + \|b\|).$$

So the classical error bounds in GADI-IR in steps 1 and 4 are hold:

$$(2.6) \quad \hat{r}^{(k)} = fl_f(b - A\hat{x}^{(k)}) = b - A\hat{x}^{(k)} + \Delta\hat{r}^{(k)},$$

$$(2.7) \quad \hat{x}^{(k+1)} = fl(\hat{x}^{(k)} + \hat{y}^{(k)}) = \hat{x}^{(k)} + \hat{y}^{(k)} + \Delta\hat{x}^{(k)},$$

where:

$$(2.8) \quad \|\Delta\hat{r}^{(k)}\| \leq u_r\|b - A\hat{x}^{(k)}\| + (1 + u_r)u_f(\|A\|\|\hat{x}^{(k)}\| + \|b\|),$$

$$(2.9) \quad \|\Delta\hat{x}^{(k)}\| \leq \varphi_2(n)u(\|\hat{x}^{(k)}\| + \|\hat{y}^{(k)}\|).$$

LEMMA 2.2 ([14]). If $\phi(n)\kappa(S)u_r < 1/2$, then $(S + F_k)$ is non singular and

$$(S + F_k)^{-1} = (I + J_k)S^{-1}$$

where

$$\|J_k\| \leq \frac{\varphi(n)\kappa(S)u_r}{1 - \phi(n)\kappa(S)u_r} < 1.$$

Then we can prove the following theorem.

THEOREM 2.3. Let Algorithm 2.1 be applied to the linear system $Ax = b$, where $A \in \mathbb{R}^{n \times n}$ is nonsingular, and assume the solver used in step 2 and 3 is backward stable. For $k \geq 0$ the computed iterate $\hat{x}_{k+1}$ satisfies

$$\|x - \hat{x}_{k+1}\| \leq \alpha_F\|x - \hat{x}_k\| + \beta_F\|x\|,$$

where

$$\begin{aligned} \alpha_F &= \lambda + (2 - \omega)\theta\hat{\kappa}(HS) + \varphi_2(n)u + 4(2 - \omega)\varphi_2(n)\hat{\kappa}(HS)u(1 + u_r)(1 + u_f) \\ &\quad + 4(2 - \omega)\hat{\kappa}(HS)u_r + 4(2 - \omega)(1 + u_r)u_f\hat{\kappa}(HS), \\ \beta_F &= \varphi_2(n)u + 4(2 - \omega)\varphi_2(n)\hat{\kappa}(HS)u(1 + u_r)u_f + 8(2 - \omega)(1 + u_r)u_f\hat{\kappa}(HS), \end{aligned}$$

and

$$\hat{\kappa}(HS) = \kappa(H)\kappa(S) \frac{\alpha\|H\| + \alpha\|S\| + 2\alpha^2}{\|H\|\|S\|}.$$

Proof. First, the error between the exact solution x and the k th iterative solution $\hat{x}^{(k+1)}$ need to be estimated. From equation (2.7) it comes:

$$x - \hat{x}^{(k+1)} = x - \hat{x}^{(k)} - \hat{y}^{(k)} - \Delta\hat{x}^{(k)},$$

and then using equation (2.2), (2.3) and (2.5), the error between the exact solution x and the k th iterative solution $\hat{x}^{(k+1)}$ can be represented as the following equation:

$$\begin{aligned} & x - \hat{x}^{(k+1)} \\ &= x - \hat{x}^{(k)} - \hat{y}^{(k)} - \Delta\hat{x}^{(k)} \\ &= x - \hat{x}^{(k)} - (S + F_k)^{-1}(2 - \omega)\alpha\hat{z}^{(k)} - \Delta\hat{x}^{(k)} \\ &= x - \hat{x}^{(k)} - (S + F_k)^{-1}(H + G_k)^{-1}(2 - \omega)\alpha\hat{r}^{(k)} - \Delta\hat{x}^{(k)} \\ &= x - \hat{x}^{(k)} - (I + J_k)S^{-1}(I + L_k)H^{-1}p\hat{r}^{(k)} - \Delta\hat{x}^{(k)} \\ &= x - \hat{x}^{(k)} - (I + J_k)S^{-1}(I + L_k)H^{-1}p(b - A\hat{x}^{(k)} + \Delta\hat{r}^{(k)}) \\ &\quad - \Delta\hat{x}^{(k)} \\ &= x - \hat{x}^{(k)} - \Delta\hat{x}^{(k)} \\ &\quad - (I + J_k)S^{-1}(I + L_k)H^{-1}Ap(x - \hat{x}^{(k)} + A^{-1}\Delta\hat{r}^{(k)}) \\ &= x - \hat{x}^{(k)} - \Delta\hat{x}^{(k)} - (HS)^{-1}Ap(x - \hat{x}^{(k)}) \\ &\quad - S^{-1}L_kH^{-1}Ap(x - \hat{x}^{(k)}) - J_kS^{-1}H^{-1}Ap(x - \hat{x}^{(k)}) \\ &\quad - J_kS^{-1}L_kH^{-1}Ap(x - \hat{x}^{(k)}) \\ &\quad - (I + J_k)S^{-1}(I + L_k)H^{-1}p\Delta\hat{r}^{(k)}. \end{aligned}$$

Considering the iterative matrix $T(\alpha, \omega)$ of GADI framework in theorem 2.1, the following equation can be obtained:

$$\begin{aligned} x - \hat{x}^{(k+1)} &= T(\alpha, \omega)(x - \hat{x}^{(k)}) - \Delta\hat{x}^{(k)} \\ &\quad - S^{-1}L_kH^{-1}Ap(x - \hat{x}^{(k)}) - J_kS^{-1}H^{-1}Ap(x - \hat{x}^{(k)}) \\ &\quad - J_kS^{-1}L_kH^{-1}Ap(x - \hat{x}^{(k)}) \\ &\quad - (I + J_k)S^{-1}(I + L_k)H^{-1}p\Delta\hat{r}^{(k)}. \end{aligned}$$

Taking the norms of both sides of the last equation and using the fact that $\|J_k\| < 1$ in lemma 2.2, we have the following inequality for the norm error of the

exact solution and the k th iterative solution:

$$\begin{aligned}
& \|x - \hat{x}^{(k+1)}\| \\
& \leq \|T(\alpha, \omega)(x - \hat{x}^{(k)})\| + \|\Delta \hat{x}^{(k)}\| \\
& \quad + p\|L_k\| \|H^{-1}\| \|S^{-1}\| \|A\| \|x - \hat{x}^{(k)}\| \\
& \quad + p\|J_k\| \|S^{-1}\| \|H^{-1}\| \|A\| \|x - \hat{x}^{(k)}\| \\
& \quad + p\|J_k\| \|L_k\| \|H^{-1}\| \|S^{-1}\| \|A\| \|x - \hat{x}^{(k)}\| \\
& \quad + p\|H^{-1}\| \|S^{-1}\| \|I + J_k\| \|I + L_k\| \|\Delta \hat{r}^{(k)}\| \\
& \leq \|T(\alpha, \omega)(x - \hat{x}^{(k)})\| + \|\Delta \hat{x}^{(k)}\| \\
& \quad + p\|H^{-1}\| \|S^{-1}\| \|A\| (\|J_k\| + \|L_k\| + \|J_k\| \|L_k\|) \|x - \hat{x}^{(k)}\| \\
& \quad + 4p\|H^{-1}\| \|S^{-1}\| \|\Delta \hat{r}^{(k)}\|.
\end{aligned}$$

To make the equation clear, let $\|J_k\| + \|L_k\| + \|J_k\| \|L_k\| = \theta$ and use equation (2.8) and (2.9), the inequality can be simplified as:

$$\begin{aligned}
& \|x - \hat{x}^{(k+1)}\| \\
(2.10) \quad & \leq \|T(\alpha, \omega)(x - \hat{x}^{(k)})\| + p\theta\|H^{-1}\| \|S^{-1}\| \|A\| \|x - \hat{x}^{(k)}\| \\
& \quad + \varphi_2(n)u(\|\hat{x}^{(k)}\| + \|\hat{y}^{(k)}\|) \\
& \quad + 4p\|H^{-1}\| \|S^{-1}\| (u_r\|A\| \|x - \hat{x}^{(k)}\| \\
& \quad + (1 + u_r)u_f(\|A\| \|\hat{x}^{(k)}\| + \|b\|)).
\end{aligned}$$

So we have the norm error estimating formula between the exact solution x and the k -th iterative solution $\hat{x}^{(k+1)}$. Next, it is necessary to estimate the norm error of the k th $\|\hat{y}^{(k)}\|$.

Using the fact that A can be splitted as $A = M + N$, and $H = \alpha I + M$, $S = \alpha I + N$, then the norm inequality exists:

$$\begin{aligned}
(2.11) \quad & \alpha\|H^{-1}\| \|S^{-1}\| \|A\| \leq \alpha\|H^{-1}\| \|S^{-1}\| \|H + S - 2\alpha I\| \\
& \leq \alpha\|H^{-1}\| \|S^{-1}\| (\|H\| + \|S\| + 2|\alpha|) \\
& \leq \kappa(H)\kappa(S) \frac{\alpha\|H\| + \alpha\|S\| + 2\alpha^2}{\|H\| \|S\|}.
\end{aligned}$$

Again, triangle inequality yields:

$$(2.12) \quad \|\hat{x}^{(k)}\| \leq \|x\| + \|\hat{x}^{(k)} - x\|,$$

then, combining (2.13) with the fact that $Ax = b$, the following inequality can be derived:

$$(2.13) \quad \|A\| \|\hat{x}^{(k)}\| + \|b\| \leq \|A\| \|x - \hat{x}^{(k)}\| + 2\|A\| \|x\|.$$

Using (2.6) and (2.8), the estimate of $\|\hat{r}^{(k)}\|$ is:

$$\begin{aligned}
& \|\hat{r}^{(k)}\| \leq \|b - A\hat{x}^{(k)}\| + \|\Delta \hat{r}^{(k)}\| \\
& \leq (1 + u_r)\|A\| \|x - \hat{x}^{(k)}\| \\
& \quad + (1 + u_r)u_f(\|A\| \|\hat{x}^{(k)}\| + \|b\|).
\end{aligned}$$

Combining the above inequalities (2.13), (2.14) and (2.15) with lemma 2.2, there exists the estimate of $\|\hat{y}^{(k)}\|$:

$$\begin{aligned}
\|\hat{y}^{(k)}\| &= p\|(S + F_k)^{-1}\|\|\hat{z}^{(k)}\| \\
&= p\|(S + F_k)^{-1}(H + G_k)^{-1}\hat{r}^{(k)}\| \\
&= p\|(I + J_k)S^{-1}(I + L_k)H^{-1}\hat{r}^{(k)}\| \\
&\leq 4p\|H^{-1}\|\|S^{-1}\|\|\hat{r}^{(k)}\| \\
&\leq 4p\|H^{-1}\|\|S^{-1}\|(\|A\|\|x - \hat{x}^{(k)}\| + u_r\|A\|\|x - \hat{x}^{(k)}\| \\
&\quad + (1 + u_r)u_f(\|A\|\|\hat{x}^{(k)}\| + \|b\|)) \\
&\leq 4p(1 + u_r)\|H^{-1}\|\|S^{-1}\|(\|A\|\|x - \hat{x}^{(k)}\|) \\
&\quad + 4p(1 + u_r)u_f\|H^{-1}\|\|S^{-1}\|(\|A\|\|\hat{x}^{(k)}\| + \|b\|) \\
&\leq 4p(1 + u_r)\|H^{-1}\|\|S^{-1}\|(\|A\|\|x - \hat{x}^{(k)}\|) \\
&\quad + 4p(1 + u_r)u_f\|H^{-1}\|\|S^{-1}\|(\|A\|\|x - \hat{x}^{(k)}\| + 2\|A\|\|x\|) \\
&= 4p(1 + u_r)(1 + u_f)\|H^{-1}\|\|S^{-1}\|\|A\|\|x - \hat{x}^{(k)}\| \\
&\quad + 8p(1 + u_r)u_f\|H^{-1}\|\|S^{-1}\|\|A\|\|x\|.
\end{aligned}
\tag{2.14}$$

Theorem 2.1 states that the radius of the iterative matrix $T(\alpha, \omega)$ is $\rho(T(\alpha, \omega))$ and $\rho(T(\alpha, \omega)) < 1$, so the following inequality can be derived:

$$\|T(\alpha, \omega)(x - \hat{x}^{(k)})\| \leq \lambda\|x - \hat{x}^{(k)}\|,$$

where $\lambda < 1$. Injecting equations (2.12), (2.13), (2.14), (2.15) and (2.16) in equation (2.11) and let $\kappa(H)\kappa(S)\frac{\alpha\|H\|+\alpha\|S\|+2\alpha^2}{\|H\|\|S\|} = \hat{\kappa}(HS)$ yields:

$$\begin{aligned}
&\|x - \hat{x}^{(k+1)}\| \\
&\leq \|T(\alpha, \omega)(x - \hat{x}^{(k)})\| + (2 - \omega)\theta\hat{\kappa}(HS)\|x - \hat{x}^{(k)}\| + \varphi_2(n)u\|x\| + \varphi_2(n)u\|x - \hat{x}^{(k)}\| \\
&\quad + 4(2 - \omega)\varphi_2(n)\hat{\kappa}(HS)u(1 + u_r)((1 + u_f)\|x - \hat{x}^{(k)}\| + 2u_f\|x\|) \\
&\quad + 4(2 - \omega)\hat{\kappa}(HS)u_r\|x - \hat{x}^{(k)}\| + 4(2 - \omega)(1 + u_r)u_f\hat{\kappa}(HS)(\|x - \hat{x}^{(k)}\| + 2\|x\|) \\
&\leq (\lambda + (2 - \omega)\theta\hat{\kappa}(HS) + \varphi_2(n)u + 4(2 - \omega)\varphi_2(n)\hat{\kappa}(HS)u(1 + u_r)(1 + u_f) \\
&\quad + 4(2 - \omega)\hat{\kappa}(HS)u_r + 4(2 - \omega)(1 + u_r)u_f\hat{\kappa}(HS))\|x - \hat{x}^{(k)}\| \\
&\quad + (\varphi_2(n)u + 4(2 - \omega)\varphi_2(n)\hat{\kappa}(HS)u(1 + u_r)u_f + 8(2 - \omega)(1 + u_r)u_f\hat{\kappa}(HS))\|x\| \\
&= \alpha_F\|x - \hat{x}^{(k)}\| + \beta_F\|x\|,
\end{aligned}
\tag{2.15}$$

where:

$$\begin{aligned}
\alpha_F &= \lambda + (2 - \omega)\theta\hat{\kappa}(HS) + \varphi_2(n)u + 4(2 - \omega)\varphi_2(n)\hat{\kappa}(HS)u(1 + u_r)(1 + u_f) \\
&\quad + 4(2 - \omega)\hat{\kappa}(HS)u_r + 4(2 - \omega)(1 + u_r)u_f\hat{\kappa}(HS), \\
\beta_F &= \varphi_2(n)u + 4(2 - \omega)\varphi_2(n)\hat{\kappa}(HS)u(1 + u_r)u_f + 8(2 - \omega)(1 + u_r)u_f\hat{\kappa}(HS). \quad \square
\end{aligned}$$

According to theorem 2.3, we have the following corollary that provides the error estimation of GADI-IR.

COROLLARY 2.4. *Let x be the exact solution of $Ax = b$ and x^* be the solution calculated by GADI-IR, then we have the following error estimation:*

$$\begin{aligned}
 (2.16) \quad \lim_{k \rightarrow \infty} \|x - \hat{x}_k\| &= \|x - x^*\| \\
 &\leq \beta_F(1 - \alpha_F)^{-1} \|x\| \\
 &= \frac{\phi_F(n)\hat{\kappa}(HS)u}{1 - \psi_F(n)\hat{\kappa}(HS)u_r} \|x\|.
 \end{aligned}$$

Proof. To make sure that the result to converge to the exact solution, it is necessary that

$$\begin{aligned}
 \alpha_F &< 1, \\
 \beta_F &< 1.
 \end{aligned}$$

Using theorem 2.3, we have

$$\begin{aligned}
 &\lim_{k \rightarrow \infty} \|x - \hat{x}_{k+1}\| \\
 &\leq \lim_{k \rightarrow \infty} (\alpha_F \|x - \hat{x}_k\| + \beta_F \|x\|) \\
 &\leq \lim_{k \rightarrow \infty} (\alpha_F^k \|x - \hat{x}_1\| + \beta_F \frac{1 - \alpha_F^k}{1 - \alpha_F} \|x\|).
 \end{aligned}$$

Note that α_F and β_F are of the form:

$$\alpha_F = \psi_F(n)\hat{\kappa}(HS)u_r, \beta_F = \varphi_2(n)u + \phi_F(n)\hat{\kappa}(HS)u_f,$$

are respectively determined by u_r and u , so x_k converges. Now we set

$$\lim_{k \rightarrow \infty} \hat{x}_k = x^*.$$

Combining (2.15) and using the form of α_F and β_F , we have

$$\begin{aligned}
 (2.17) \quad \lim_{k \rightarrow \infty} \|x - \hat{x}_k\| &= \|x - x^*\| \\
 &\leq \beta_F(1 - \alpha_F)^{-1} \|x\| \\
 &= \frac{\varphi_2(n)u + \phi_F(n)\hat{\kappa}(HS)u_f}{1 - \psi_F(n)\hat{\kappa}(HS)u_r} \|x\|.
 \end{aligned} \quad \square$$

From corollary 2.4, it can be seen that the term α_F is the rate of convergence and depends on the condition number of the matrix H, S , and the precision used u_r . The term β_F is the limiting accuracy of the method and depends on the precision accuracy used u .

2.3. Backward analysis.

LEMMA 2.5 ([14]). *If $\mu(n)\kappa(S)u_r < 1/2$, then $(H + F_k)$ is non singular and*

$$(2.18) \quad (S + F_k)^{-1} = S^{-1}(I + P_k),$$

where:

$$(2.19) \quad \|P_k\| \leq \frac{\mu(n)\kappa(S)u_r}{1 - \mu(n)\kappa(S)u_r} \leq 1.$$

THEOREM 2.6. *Let Algorithm 2.1 be applied to a linear system $Ax = b$ with a nonsingular matrix $A \in \mathbb{R}^{n \times n}$ and assume the solver used in step 2 and 3 is backward stable. Then for $k \geq 0$ the computed iterate $\hat{x}_{k+1}$ satisfies*

$$(2.20) \quad \frac{\|b - Ax_{k+1}\|}{\|A\|\|x_{k+1}\|} \leq \alpha_B \frac{\|b - Ax_k\|}{\|A\|\|x_k\|} + \beta_B,$$

where:

$$\begin{aligned} \alpha_B &= \gamma(\lambda + (2 - \omega)\eta\hat{\kappa}(HS) + 4(2 - \omega)\hat{\kappa}(HS)u_r + 4(2 - \omega)\hat{\kappa}(HS)(1 + u_r)u_f), \\ \beta_B &= 8(2 - \omega)\gamma\hat{\kappa}(HS)(1 + u_r)u_f + \varphi_2(n)\gamma u \\ &\quad + \varphi_2(n)(1 - \varphi_2(n)u)^{-1}u(1 + (1 + \varphi_2(n)u)\gamma). \end{aligned}$$

and

$$\hat{\kappa}(HS) = \kappa(H)\kappa(S) \frac{\alpha\|H\| + \alpha\|S\| + 2\alpha^2}{\|H\|\|S\|}.$$

Proof. Building upon equations (2.2) and (2.3), we can derive the following sequence of equations:

$$\begin{aligned} & x - \hat{x}^{(k+1)} \\ &= x - \hat{x}^{(k)} - \hat{y}^{(k)} - \Delta\hat{x}^{(k)} \\ &= x - \hat{x}^{(k)} - (S + F_k)^{-1}p\hat{z}^{(k)} - \Delta\hat{x}^{(k)} \\ &= x - \hat{x}^{(k)} - p(S + F_k)^{-1}(H + G_k)^{-1}\hat{r}^{(k)} - \Delta\hat{x}^{(k)}. \end{aligned}$$

Subsequently, by applying equation (2.5) for the residual computation and leveraging the key result from lemma 2.5 regarding the inverse matrix structure, we can derive:

$$\begin{aligned} & x - \hat{x}^{(k+1)} \\ &= x - \hat{x}^{(k)} - pS^{-1}(I + P_k)H^{-1}(I + Q_k)\hat{r}^{(k)} - \Delta\hat{x}^{(k)} \\ &= x - \hat{x}^{(k)} - \Delta\hat{x}^{(k)} \\ &\quad - pS^{-1}(I + P_k)H^{-1}(I + Q_k)(b - A\hat{x}^{(k)} + \Delta\hat{r}^{(k)}). \end{aligned}$$

To obtain an expression for the residual, we multiply both sides of the equation by the coefficient matrix A on the left, which yields:

$$\begin{aligned} & b - A\hat{x}^{(k+1)} \\ &= b - A\hat{x}^{(k)} - A\Delta\hat{x}^{(k)} \\ &\quad - pAS^{-1}(I + P_k)H^{-1}(I + Q_k)(b - A\hat{x}^{(k)} + \Delta\hat{r}^{(k)}) \\ &= b - A\hat{x}^{(k)} - pAS^{-1}(I + P_k)H^{-1}(I + Q_k)(b - A\hat{x}^{(k)}) \\ &\quad - pAS^{-1}(I + P_k)H^{-1}(I + Q_k)\Delta\hat{r}^{(k)} - A\Delta\hat{x}^{(k)} \\ &= T(\alpha, \omega)(b - A\hat{x}^{(k)}) - A\Delta\hat{x}^{(k)} \\ &\quad - S^{-1}P_kH^{-1}Ap(b - A\hat{x}^{(k)}) - Q_kS^{-1}H^{-1}Ap(b - A\hat{x}^{(k)}) \\ &\quad - Q_kS^{-1}P_kH^{-1}Ap(b - A\hat{x}^{(k)}) \\ &\quad - A(I + Q_k)S^{-1}(I + P_k)H^{-1}p\Delta\hat{r}^{(k)}. \end{aligned}$$

Taking the norm of both sides and using the fact that $\|P_k\| < 1$ and letting $\|P_k\| + \|Q_k\| + \|P_k\|\|Q_k\| = \eta$ again gives:

$$\begin{aligned}
& \|b - A\hat{x}^{(k+1)}\| \\
& \leq \|T(\alpha, \omega)(b - A\hat{x}^{(k)})\| \\
& \quad + p\|S^{-1}\|\|P_k\|\|H^{-1}\|\|A\|\|b - A\hat{x}^{(k)}\| \\
& \quad + p\|Q_k\|\|S^{-1}\|\|H^{-1}\|\|A\|\|b - A\hat{x}^{(k)}\| \\
& \quad + p\|Q_k\|\|S^{-1}\|\|P_k\|\|H^{-1}\|\|A\|\|b - A\hat{x}^{(k)}\| \\
& \quad + p\|A\|\|S^{-1}\|\|H^{-1}\|\|I + P_k\|\|I + Q_k\|\|\Delta\hat{r}^{(k)}\| \\
& \quad + \|A\|\|\Delta\hat{x}^{(k)}\| \\
& \leq \|T(\alpha, \omega)(b - A\hat{x}^{(k)})\| + p\eta\|H^{-1}\|\|S^{-1}\|\|b - A\hat{x}^{(k)}\| \\
& \quad + 4p\|H^{-1}\|\|S^{-1}\|\|A\|\|\Delta\hat{r}^{(k)}\| + \|A\|\|\Delta\hat{x}^{(k)}\|.
\end{aligned}$$

Applying Equations (2.8) and (2.9) to further refine our analysis, we obtain the following expression:

$$\begin{aligned}
& \|b - A\hat{x}^{(k+1)}\| \\
(2.21) \quad & \leq \|T(\alpha, \omega)(b - A\hat{x}^{(k)})\| + p\eta\|H^{-1}\|\|S^{-1}\|\|A\|\|b - A\hat{x}^{(k)}\| \\
& \quad + 4p\|H^{-1}\|\|S^{-1}\|\|A\|(\|u_r\|\|b - A\hat{x}^{(k)}\| \\
& \quad + (1 + u_r)u_f(\|A\|\|\hat{x}^{(k)}\| + \|b\|)) \\
& \quad + \varphi_2(n)u\|A\|(\|\hat{x}^{(k)}\| + \|\hat{y}^{(k)}\|).
\end{aligned}$$

For $\|b\|$ according to $b = b - A\hat{x}^{(k)} + A\hat{x}^{(k)}$ we have:

$$(2.22) \quad \|b\| \leq \|b - A\hat{x}^{(k)}\| + \|A\|\|\hat{x}^{(k)}\|.$$

Then we need to calculate the $\|\hat{y}^{(k)}\|$ in equation (2.20), using equation (2.7) and (2.9):

$$\begin{aligned}
\|\hat{y}^{(k)}\| &= \|\hat{x}^{(k+1)} - \hat{x}^{(k)} - \Delta\hat{x}^{(k)}\| \\
&\leq \|\hat{x}^{(k+1)}\| + \|\hat{x}^{(k)}\| + \|\Delta\hat{x}^{(k)}\| \\
&\leq \|\hat{x}^{(k+1)}\| + \|\hat{x}^{(k)}\| + \varphi_2(n)u\|\hat{x}^{(k)}\| + \varphi_2(n)u\|\hat{y}^{(k)}\|
\end{aligned}$$

then:

$$(2.23) \quad \|\hat{y}^{(k)}\| \leq (1 - \varphi_2(n)u)^{-1}(\|\hat{x}^{(k+1)}\| + (1 + \varphi_2(n)u)\|\hat{x}^{(k)}\|).$$

Finally, injecting equations (2.11), (2.21) and (2.22) in equation (2.20) yields:

$$\begin{aligned}
\|b - Ax_{k+1}\| &\leq \lambda\|b - Ax_k\| + (2 - \omega)\eta\hat{\kappa}(HS)\|b - A\hat{x}^{(k)}\| \\
&\quad + 4(2 - \omega)\hat{\kappa}(HS)u_r\|b - A\hat{x}^{(k)}\| \\
&\quad + 4(2 - \omega)\hat{\kappa}(HS)(1 + u_r)u_f\|A\|\|\hat{x}^{(k)}\| \\
&\quad + 4(2 - \omega)\hat{\kappa}(HS)(1 + u_r)u_f(\|b - A\hat{x}^{(k)}\| + \|A\|\|\hat{x}^{(k)}\|) \\
&\quad + \varphi_2(n)u\|A\|\|\hat{x}^{(k)}\| \\
&\quad + \varphi_2(n)(1 - \varphi_2(n)u)^{-1}u\|A\|(\|\hat{x}^{(k+1)}\| + (1 + \varphi_2(n)u)\|\hat{x}^{(k)}\|).
\end{aligned}$$

From forward analysis note that there exists γ so that $\|\hat{x}^{(k)}\| \leq \gamma \|\hat{x}^{(k+1)}\|$, then:

$$\begin{aligned}
& \frac{\|b - Ax_{k+1}\|}{\|A\| \|x_{k+1}\|} \\
& \leq \gamma(\lambda + (2 - \omega)\eta\hat{\kappa}(HS) + 4(2 - \omega)\hat{\kappa}(HS)u_r + 4(2 - \omega)\hat{\kappa}(HS)(1 + u_r)u_f) \frac{\|b - Ax_k\|}{\|A\| \|x_k\|} \\
& \quad + 8(2 - \omega)\gamma\hat{\kappa}(HS)(1 + u_r)u_f + \varphi_2(n)\gamma u \\
& \quad + \varphi_2(n)(1 - \varphi_2(n)u)^{-1}u(1 + (1 + \varphi_2(n)u)\gamma) \\
& = \alpha_B \frac{\|b - Ax_k\|}{\|A\| \|x_k\|} + \beta_B,
\end{aligned}$$

where:

$$\begin{aligned}
\alpha_B &= \gamma(\lambda + (2 - \omega)\eta\hat{\kappa}(HS) + 4(2 - \omega)\hat{\kappa}(HS)u_r + 4(2 - \omega)\hat{\kappa}(HS)(1 + u_r)u_f), \\
\beta_B &= 8(2 - \omega)\gamma\hat{\kappa}(HS)(1 + u_r)u_f + \varphi_2(n)\gamma u \\
& \quad + \varphi_2(n)(1 - \varphi_2(n)u)^{-1}u(1 + (1 + \varphi_2(n)u)\gamma). \quad \square
\end{aligned}$$

It can be seen that the term α_B is the rate of convergence and depends on the condition number of the matrix H, S and parameter α , and the precision used u_r . The term β_B is the limiting accuracy of the method and depends on the precision accuracy used u .

3. Parameter prediction and regularization.

3.1. Regularization. The regularization parameter α plays a crucial role in determining the performance and stability of GADI. In this section, we analyze how to optimally select the regularization parameter α to effectively balance the splitting matrices and ensure robust convergence in the inner low-precision steps of GADI-IR. We focus particularly on its impact on numerical stability and convergence behavior when operating in mixed-precision environments.

In Algorithm 2.1, inner loop step 2 and step 3 are performed with coefficient matrix $H = \alpha I + H, S = \alpha I + S$ which are of the form:

$$(3.1) \quad \alpha I + U,$$

with regularization parameter α .

For the regularized matrix in (3.1), we can explicitly compute its condition number. The 2-norm condition number of matrix $\alpha I + U$ is given by:

$$\begin{aligned}
(3.2) \quad \kappa_2(\alpha I + U) &= \frac{\sigma_{\max}(\alpha I + U)}{\sigma_{\min}(\alpha I + U)} \\
&= \frac{\alpha + \sigma_{\max}(U)}{\alpha + \sigma_{\min}(U)},
\end{aligned}$$

where $\sigma_{\max}$ and $\sigma_{\min}$ denote the largest and smallest singular values respectively. This expression reveals an important property: as the regularization parameter α increases, the ratio between the maximum and minimum singular values decreases, thereby improving the condition number of the regularized matrix $\alpha I + U$.

Then, we can analyse the $\hat{\kappa}_2(HS)$, for $\hat{\kappa}_2(HS)$ we have:

$$\begin{aligned}
 \hat{\kappa}_2(HS) &= \kappa_2(H)\kappa_2(S) \frac{\alpha\|H\|_2 + \alpha\|S\|_2 + 2\alpha^2}{\|H\|_2\|S\|_2} \\
 &= \kappa_2(H)\kappa_2(S) \frac{\alpha\|\alpha I + N\|_2 + \alpha\|\alpha I + M\|_2 + 2\alpha^2}{\|\alpha I + M\|_2\|\alpha I + N\|_2} \\
 (3.3) \quad &= \frac{\alpha(\alpha + \sigma_{\max}(M) + \alpha + \sigma_{\max}(N)) + \alpha^2}{(\alpha + \sigma_{\min}(M))(\alpha + \sigma_{\min}(N))} \\
 &= \frac{4\alpha^2 + \alpha(\sigma_{\max}(M) + \sigma_{\max}(N))}{\alpha^2 + \alpha(\sigma_{\min}(M) + \sigma_{\min}(N)) + \sigma_{\min}(M)\sigma_{\min}(N)}.
 \end{aligned}$$

It is obvious from (3.3) that $\hat{\kappa}_2(HS)$ is a monotonically decreasing function with respect to α . Also, it is clearly from (3.3) that:

$$\lim_{\alpha \rightarrow \infty} \hat{\kappa}_2(HS) = 4.$$

This sensitivity to α becomes particularly pronounced in mixed-precision environments, where reduced precision operations during iteration can amplify small errors, especially if the regularization term is not well-calibrated. Therefore, determining an optimal value for α is crucial for maintaining the robustness of the mixed-precision GADI-IR algorithm, as it ensures that the computational efficiency gains are achieved without compromising solution accuracy or convergence reliability.

3.2. Backward analysis for regularization. In this section, 2-Norm will be used as the symbolic norm to satisfy (3.2). Theorem 2.6 provides the backward error analysis of GADI-IR, where α_B determines the convergence rate and β_B characterizes the ultimate achievable accuracy of the method.

According to [5], for a given matrix U , reducing its precision can lead to an improvement in its condition number. When down-casting a matrix U (e.g., from double precision to single precision), its smallest singular value increases while the largest singular value remains largely unchanged which can be expressed mathematically as:

$$(3.4) \quad \kappa_2(U_r) \leq \kappa_2(U),$$

where U_r denotes the reduced-precision representation of matrix U stored with precision u_r . However, this improvement is not so significant.

For the full precision GADI algorithm where all computational steps are performed in high precision, the convergence analysis has been established in [10]. Specifically, for α_B in Theorem 2.6, we have:

$$\begin{aligned}
 \hat{\alpha}_B &= \gamma(\lambda + (2 - \omega)\eta\hat{\kappa}_2(HS) + 4(2 - \omega)\hat{\kappa}_2(HS)u + 4(2 - \omega)\hat{\kappa}_2(HS)(1 + u)u) \\
 &= \gamma\lambda + (2 - \omega)\hat{\kappa}_2(HS)(\|P_k\| + \|Q_k\| + \|P_k\|\|Q_k\| + 4u + 4(1 + u)u),
 \end{aligned}$$

where we consider the case of GADI with uniform precision $u = u_f = u_r$.

For low-precision computations in GADI-IR, the coefficient α_B from Theorem 2.6 can be expressed as:

$$\begin{aligned}
 \alpha_B &= \gamma(\lambda + (2 - \omega)\eta\hat{\kappa}_2(H_r S_r) + 4(2 - \omega)\hat{\kappa}_2(H_r S_r)u_r + 4(2 - \omega)\hat{\kappa}_2(H_r S_r)(1 + u_r)u_f) \\
 (3.5) \quad &= \gamma\lambda + (2 - \omega)\hat{\kappa}_2(H_r S_r)(\|P_k\| + \|Q_k\| + \|P_k\|\|Q_k\| + 4u_r + 4(1 + u_r)u_f),
 \end{aligned}$$

where H_r and S_r represent the reduced-precision versions of matrices H and S respectively, both stored with precision u_r , satisfying $u \leq u_f \leq u_r$. Based on equation (3.5) and Lemma 2.5, we can establish:

$$(3.6) \quad \alpha_B = f(\alpha, u_r, u_f),$$

where $f(\alpha, u_r, u_f)$ is a function dependent on α , u_r , and u_f . Based on (3.3) and (3.5) and considering that the improvement in (3.4) is not so significant, then $f(\alpha, u_r, u_f)$ exhibits monotonic behavior - decreasing with respect to α while increasing with respect to both u_r and u_f . If we fix α and temporarily disregard precision's influence on the matrix condition number because the improvement in (3.4) is not so significant by setting $\hat{\kappa}_2(HS) = \hat{\kappa}_2(H_r S_r)$, we obtain:

$$\alpha_B = f(\alpha, u_r, u_f) \geq \hat{\alpha}_B = f(\alpha, u, u) \leq 1,$$

for $u_r \geq u, u_f \geq u$.

Consequently, α_B in GADI-IR could potentially exceed 1, leading to algorithmic divergence. Therefore, to ensure convergence of GADI-IR when $u \leq u_f \leq u_r$, it is essential to maintain $\alpha_B < 1$, which according to (3.3) necessitates a larger value of α .

In summary, while using lower precision for matrix H can improve its condition number and potentially enhance the convergence rate of GADI-IR, this reduction in precision introduces larger values of u_r and u_f in equation (3.5), which may result in an increased $\alpha_B > 1$. For GADI-IR to converge, it is crucial that α_B remains less than 1. However, the larger α_B resulting from lower precision could cause α_B to exceed 1, leading to algorithmic divergence. Therefore, it becomes essential to employ the regularization parameter α to achieve an even smaller $\hat{\kappa}(H_r S_r)$, thereby ensuring α_B stays below 1 and maintaining convergence.

3.3. Parameter prediction. Paper[10] shows that the parameter α is important to the performance of GADI. In this section, we will use mixed precision to accelerate the parameter prediction of GADI-IR.

3.3.1. Parameter prediction in GADI. The performance of GADI is sensitive to the splitting parameters. Paper[10] proposed a data-driven parameter selection method, the Gaussian Process Regression (GPR) approach based on the Bayesian inference, which can efficiently obtain accurate splitting parameters. The Gaussian Process Regression (GPR) prediction process is illustrated in Figure 1

From Figure 1, it can be seen that the Gaussian Process Regression (GPR) method established a mapping between the matrix size n and the parameter α . By a series known data of parameters, we can predict unknown parameters. The known relatively optimal parameters in the training data set come from small-scale linear systems, while the unknown parameter belongs to that of large linear systems. The predicted data in the training set is also used to form the retraining set to predict the parameter more accurately and extensively.

3.3.2. Parameter prediction training set. As we use Gaussian Process Regression (GPR) prediction to predict the parameter α , we need to first get the training set. To get the training set, it is necessary for us to analysis the structure of linear system automatically to construct a series of small linear systems with the same structure which will take a lot of time. To reduce the time consumption, we put this progress in FP32 low precision. Then we will get a series of small linear systems with the same

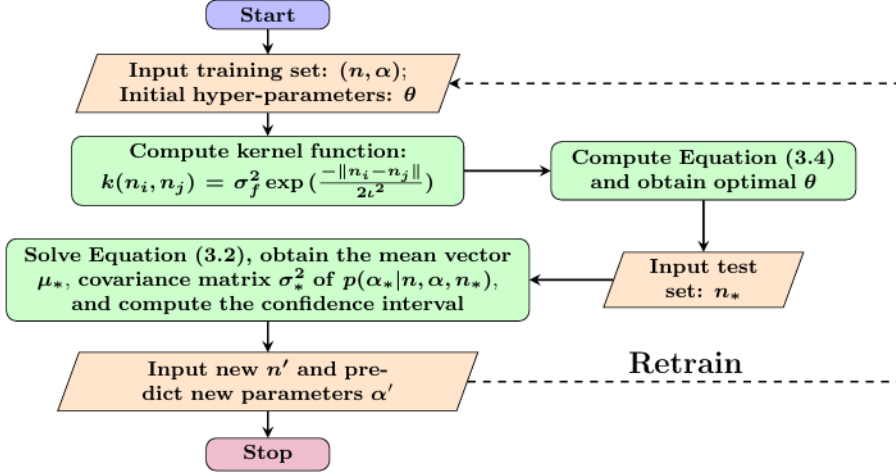


Fig. 1: Flow chart of Gaussian Process Regression (GPR) parameters prediction.

structure of the original linear system. Afterwards we use the dichotomy to find a series of $\{\alpha_k\}$ with these low scale linear systems in FP32 precision.

Finally, we will use the calculated training set $\{\alpha_k\}$ to do Gaussian Process Regression (GPR) prediction with FP32 precision.

By using low precision to find the training set, we can get the best parameter α with less time of the original implementation without losing performance in GADI-IR with parameter prediction which is illustrated in Table 2. The result of Table 2 is tested on experiment three-dimensional convection-diffusion equation in section 4.1.

n^3	α FP64	α FP32
32^3	0.0699	0.0699
64^3	0.0599	0.0599
128^3	0.0595	0.0595

Table 2: α predicted by different precision training set for three-dimensional convection-diffusion equation.

4. Numerical Experiments. In this section, we will evaluate our algorithm using various mixed precision configurations to validate our analysis.

4.1. Three-dimensional convection-diffusion equation. Consider 3D convection-diffusion equation:

$$(4.1) \quad -(u_{x_1 x_1} + u_{x_2 x_2} + u_{x_3 x_3}) + (u_{x_1} + u_{x_2} + u_{x_3}) = f(x_1, x_2, x_3)$$

on the unit cube $\Omega = [0, 1] \times [0, 1] \times [0, 1]$ with Dirichlet boundary condition. By using the centered difference method to discretize the convective-diffusion equation, we can obtain the linear sparse system $Ax = b$. The coefficient matrix A is :

$$A = T_x \otimes I \otimes I + I \otimes T_y \otimes I + I \otimes I \otimes T_z,$$

where T_x, T_y, T_z are the tridiagonal matrices. $T_x = \text{Tridiag}(t_2, t_1, t_3)$, $T_y = T_z = \text{Tridiag}(t_2, 0, t_3)$, $t_1 = 6$, $t_2 = -1 - r$, $t_3 = -1 + r$, $r = 1/(2n + 2)$. n is the degree of freedom in each direction. $x \in \mathbb{R}^{n^3}$ is the solution vector and $b \in \mathbb{R}^{n^3}$ is the right-hand side vector which is generated by choosing the exact solution $x = (1, 1, \dots, 1)^T$. The relative error is defined as $\text{RES} = \|r^{(k)}\|_2 / \|r^{(0)}\|_2$. All tests are started from the zero vector. $r^{(k)} = b - Ax^{(k)}$ is the k -th step residual.

The GADI-IR algorithm is tested on the 3D convection-diffusion equation with

$$(4.2) \quad H = \frac{A + A^*}{2}, S = \frac{A - A^*}{2}.$$

[3] splitting strategy. The results of the numerical experiments are shown in Table 3. The table presents the relative residuals (RRES) for the 3D convection-diffusion equation using different combinations of precisions for the components u_r , u , and u_f . The experiments demonstrate that using double precision for all components consistently achieves the lowest residuals. In contrast, using half precision for u_r results in higher residuals. This outcome aligns with our error analysis of GADI-IR in section 2, which predicts the convergence of GADI-IR and increased sensitivity and potential instability when lower precision is employed for critical computations particularly when α is set to lower values. The results highlight the sensitivity of the algorithm's performance to the choice of precision and the regularization parameter α .

u_r	u	u_f	α	RRES
double	double	double	0.01	10^{-13}
single	single	single	0.01	10^{-4}
single	double	double	0.01	10^{-13}
half	double	single	0.01	—
half	double	double	0.01	—
half	single	single	0.01	10^{-4}
double	double	double	0.02	10^{-13}
single	single	single	0.02	10^{-4}
single	double	double	0.02	10^{-13}
half	double	single	0.02	10^{-8}
half	double	double	0.02	10^{-8}
half	single	single	0.02	10^{-4}
double	double	double	10.0	10^{-13}
single	single	single	10.0	10^{-4}
single	double	double	10.0	10^{-13}
half	double	single	10.0	10^{-10}
half	double	double	10.0	10^{-10}
half	single	single	10.0	10^{-4}

Table 3: *Relative Residual with different precisions for 3D convection-diffusion equation.*

u_r	u	u_f	α	RRES
double	double	double	0.01	10^{-9}
single	single	single	0.01	10^{-2}
single	double	double	0.01	10^{-9}
half	double	single	0.01	—
half	double	double	0.01	—
half	single	single	0.01	10^{-3}
double	double	double	0.02	10^{-9}
single	single	single	0.02	10^{-2}
single	double	double	0.02	10^{-9}
half	double	single	0.02	10^{-5}
half	double	double	0.02	10^{-5}
half	single	single	0.02	10^{-3}
double	double	double	10.0	10^{-9}
single	single	single	10.0	10^{-2}
single	double	double	10.0	10^{-9}
half	double	single	10.0	10^{-6}
half	double	double	10.0	10^{-6}
half	single	single	10.0	10^{-3}

Table 4: *Relative Residual with different precisions for CARE.*

From Table 3, It can be seen that the mixed precision algorithm GADI-IR with

$u_r = \text{half}$, $u = u_f = \text{double}$ exhibits significant sensitivity to the regularization parameter α . This mixed precision strategy, where the reduced precision (FP16) is used for the iterative process and the higher precision (double) is retained for key updates and parameter calculations, provides considerable performance benefits in terms of computation speed and memory usage. However, the choice of α , which controls the balance between the regularization and the solution accuracy, plays a crucial role in ensuring the stability and convergence of the algorithm.

Based on this mixed precision strategy ($u_r = \text{half}$, $u = u_f = \text{double}$), we conducted experiments to investigate the impact of varying α on the convergence residual RES. The results, as shown in the Figure 2 and 3, illustrate a clear trend: as the regularization parameter α increases, the convergence performance improves for this specific test case.

This behavior indicates that larger values of α enhance the stability of the iterative process, reducing the impact of precision-related errors and promoting more reliable convergence to the desired solution. The experimental results suggest that α effectively mitigates the potential inaccuracies introduced by the mixed precision approach just as the theoretical analysis of regularization in section 3.1 and 3.2 predicted.

While increasing the regularization parameter α generally improves the convergence residual RES, it may also lead to a higher number of iterations required for convergence. This is because larger values of α can over-regularize the system, effectively damping the iterative process and slowing down the overall rate of convergence. Table 5 shows how different values of the regularization parameter α influence the residual RES and the number of iteration steps. Smaller or larger α values lead to a significant increase in iteration steps, while moderate α values result in fewer steps and smaller residuals. This indicates that the choice of α is crucial for balancing efficiency and accuracy.

Alpha (α)	Residual (res)	Iteration Steps
0.01	—	—
0.02	4.86e-08	2126
0.05	3.10e-08	465
0.1	1.88e-08	150
0.5	4.85e-09	75
1	2.47e-09	120
5	4.96e-10	468
10	3.51e-10	909
100	3.51e-11	8862

Table 5: *Impact of Regularization Parameter α on Convergence Residuals and Iteration Steps.*

As a result, selecting the optimal α involves balancing the trade-off between minimizing the residual error and controlling the computational cost associated with additional iterations. For practical applications, it is essential to choose α based on the specific problem characteristics and acceptable computational overhead.

By carefully tuning α within a reasonable range, one can achieve a compromise that maintains a low residual error while avoiding excessive iteration counts, thereby

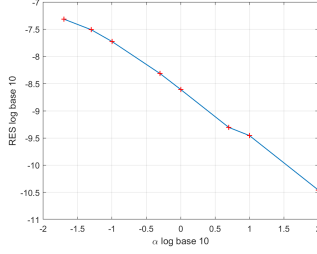


Fig. 2: *Impact of α on convergence residual of 3d convection-diffusion equation.*

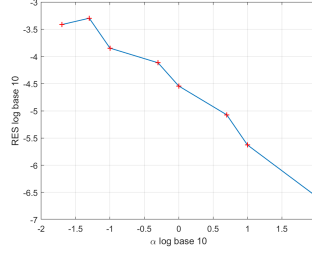


Fig. 3: *Impact of α on convergence residual of CARE.*

optimizing both accuracy and efficiency under the constraints of the mixed precision strategy. Adaptive or problem-specific strategies for determining α could further enhance the robustness and practicality of the algorithm in diverse scenarios.

4.2. CARE equation. Next, we apply our algorithm to other problem. We test our algorithm on CARE equation which is a typical problem in control theory. In this section, we consider the continuous-time algebraic Riccati equation(CARE):

$$(4.3) \quad A^T X + X A - X K X + Q = 0.$$

where $A, K, Q \in \mathbb{C}^{n \times n}$, $K = K^*$, $Q = Q^*$, and X is an unknown matrix. Paper [9] proposed an algorithm named Newton-GDAI which is based on GADI to solve this equation. In this algorithm, GADI is used to solve Lyapunov equation in inner iteration where we apply our mixed precision algorithm GADI-IR. In this equation, complex matrices are used where we follow the work of [1] to handle complex arithmetic in half precision. Their research provides effective strategies for mixed-precision computation with complex matrices on GPUs, which is essential for our implementation. The results of the numerical experiments, as shown in Table 4, indicate that the mixed precision algorithm GADI-IR demonstrates varying levels of residual error (RRES) depending on the precision levels used for different components (u_r, u, u_f) and the regularization parameter α . Specifically, using double precision for all components consistently achieves the lowest residual errors across different values of α . In contrast, using half precision for u_r results in higher residual errors, particularly when α is set to lower values. The experiments highlight the sensitivity of the algorithm's performance to the choice of precision and the regularization parameter, emphasizing the need for careful selection to balance computational efficiency and accuracy.

Similarly, the impact of the regularization parameter α on the convergence residual RES for the mixed precision strategy with $u = u_f = \text{double}$ and $u_r = \text{single}$ in the CARE problem is illustrated in Figure 3, akin to the analysis for the 3D convection-diffusion equation discussed in section 4.1.

4.3. Sylvester equation. To further test the performance of our algorithm, we apply mixed GADI-IR method to continuous Sylvester equation[4]. The continuous Sylvester equation can be written as:

$$(4.4) \quad AX + XB + C = 0.$$

where $A \in \mathbb{C}^{m \times m}$, $B \in \mathbb{C}^{n \times n}$ and $C \in \mathbb{C}^{m \times n}$, are sparse matrices. $X \in \mathbb{C}^{m \times n}$ is the unknown matrix. Applying the mixed precision algorithm GADI-IR to continuous Sylvester equation and replacing splitting matrices M, N with A, B respectively, we can obtain the mixed GADI-AB method.

The sparse matrices A, B have the following structure:

$$A = B = M + 2rN + \frac{100}{(n+1)^2}I.$$

where r is a parameter which controls Hermitian dominated or skew-Hermitian dominated of matrix. $M, N \in \mathbb{C}^{n \times n}$ are tridiagonal matrices $M = \text{Tridiag}(-1, 2, -1)$, $N = \text{Tridiag}(0.5, 0, -0.5)$. We apply mixed GADI-AB to solve the Sylvester equations for $r = 0.01, 0.1, 1$, RES is calculated as $R^{(k)} = C - AX^{(k)} - X^{(k)}B$.

The numerical experiments are shown in Table 6 which presents numerical test results for the Sylvester equation under different parameter combinations of r and α , with solution accuracy evaluated by the residual (res). The results show that the residuals range from 1e-8 to 1e-10, indicating high numerical accuracy of the solutions. The method demonstrates stability and reliability across various parameter settings.

Additionally, the impact of the regularization parameter α on the convergence residual of the Sylvester equation with different r values is illustrated in Figure 4. It can be observed that the convergence residual decreases as α increases, with the rate of convergence improving for larger α values. This behavior is consistent with the results of the previous tests, indicating that the regularization parameter α plays a crucial role in balancing the precision-related errors and promoting the convergence of the mixed precision GADI-IR algorithm.

r	α	Residual (res)
0.01	0.01	—
	0.02	1.563e-08
	10	2.4092e-10
0.1	0.01	—
	0.02	1.563e-08
	10	4.2351e-10
1	0.01	—
	0.02	1.563e-08
	10	3.2551e-10

Table 6: Test results of the Sylvester equation for different r and α values.

5. Conclusion and Future work. In this paper, we have presented a novel mixed-precision iterative refinement algorithm, GADI-IR, which effectively combines multiple precision arithmetic to solve large-scale sparse linear systems. Our key contributions and findings can be summarized as follows:

First, we developed a theoretical framework for analyzing the convergence of mixed-precision GADI-IR, establishing the relationship between precision levels and convergence conditions. Through careful backward error analysis, we demonstrated how the regularization parameter α can be used to ensure convergence when using reduced precision arithmetic.

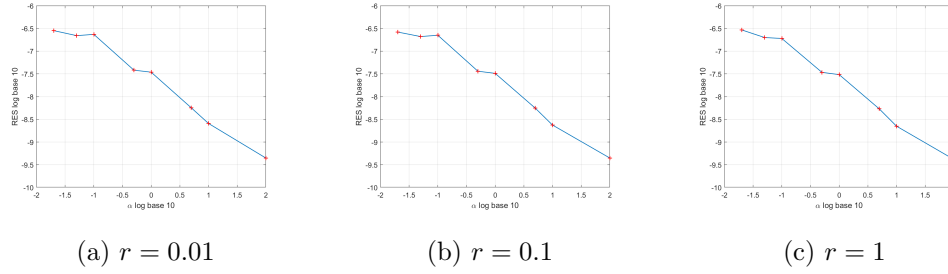


Fig. 4: Impact of α on convergence residual of Sylvester equation with different r .

Second, we successfully integrated low-precision computations into the parameter prediction process using Gaussian Process Regression (GPR), achieving significant speedup without compromising accuracy. Our experimental results showed that using FP32 for the training set generation reduces the computational time by approximately 50% while maintaining prediction quality comparable to FP64.

Third, comprehensive numerical experiments on various test problems, including three-dimensional convection-diffusion equations and Sylvester equations, validated both our theoretical analysis and the practical effectiveness of the mixed-precision approach. The results demonstrated that GADI-IR can achieve substantial acceleration while maintaining solution accuracy through appropriate precision mixing strategies.

Looking ahead, we have several promising directions for future research like investigation of newer precision formats (e.g., FP8) that could potentially further enhance the algorithm's performance and efficiency and implementation of GADI-IR on modern supercomputing platforms.

These future developments will further strengthen the practical applicability and efficiency of the GADI-IR algorithm in solving large-scale linear systems.

REFERENCES

- [1] A. ABDELFAH, S. TOMOV, AND J. DONGARRA, *Towards half-precision computation for complex matrices: A case study for mixed precision solvers on gpus*, in 2019 IEEE/ACM 10th Workshop on Latest Advances in Scalable Algorithms for Large-Scale Systems (ScalA), 2019, pp. 17–24, <https://doi.org/10.1109/ScalA49573.2019.00008>.
- [2] P. AMESTOY, A. BUTTARI, N. J. HIGHAM, J.-Y. L'EXCELLENT, T. MARY, AND B. VIEUBLÉ, *Five-precision gmres-based iterative refinement*, SIAM Journal on Matrix Analysis and Applications, 45 (2024), pp. 529–552, <https://doi.org/10.1137/23M1549079>, <https://doi.org/10.1137/23M1549079>, <https://arxiv.org/abs/https://doi.org/10.1137/23M1549079>.
- [3] Z.-Z. BAI, G. GOLUB, AND K. NG, *On successive-overrelaxation acceleration of the hermitian and skew-hermitian splitting iterations*, Numerical Linear Algebra with Applications, 14 (2007), <https://doi.org/10.1002/nla.517>.
- [4] P. BENNER, R.-C. LI, AND N. TRUHAR, *On the adi method for sylvester equations*, Journal of Computational and Applied Mathematics, 233 (2009), pp. 1035–1045, <https://doi.org/https://doi.org/10.1016/j.cam.2009.08.108>, <https://www.sciencedirect.com/science/article/pii/S0377042709006050>.
- [5] C. BOUTSIKAS, P. DRINEAS, AND I. C. F. IPSEN, *Small singular values can increase in lower precision*, SIAM Journal on Matrix Analysis and Applications, 45 (2024), pp. 1518–1540, <https://doi.org/10.1137/23M1557209>, <https://doi.org/10.1137/23M1557209>, <https://arxiv.org/abs/https://doi.org/10.1137/23M1557209>.
- [6] E. CARSON AND N. J. HIGHAM, *Accelerating the solution of linear systems by iterative refinement in three precisions*, SIAM Journal on Scientific Computing, 40 (2018), pp. A817–

- A847, <https://doi.org/10.1137/17M1140819>, <https://doi.org/10.1137/17M1140819>, <https://arxiv.org/abs/https://doi.org/10.1137/17M1140819>.
- [7] E. CARSON, N. J. HIGHAM, AND S. PRANESH, *Three-precision gmres-based iterative refinement for least squares problems*, SIAM Journal on Scientific Computing, 42 (2020), pp. A4063–A4083, <https://doi.org/10.1137/20M1316822>, <https://doi.org/10.1137/20M1316822>, <https://arxiv.org/abs/https://doi.org/10.1137/20M1316822>.
 - [8] IEEE, *Ieee standard for floating-point arithmetic*, IEEE Std 754-2019 (Revision of IEEE 754-2008), (2019), pp. 1–84, <https://doi.org/10.1109/IEEESTD.2019.8766229>.
 - [9] K. JIANG, S. LI, AND J. ZHANG, *A general alternating-direction implicit newton method for solving continuous-time algebraic riccati equation*, Applied Numerical Mathematics, 207 (2025), pp. 642–656, <https://doi.org/https://doi.org/10.1016/j.apnum.2024.09.029>, <https://www.sciencedirect.com/science/article/pii/S0168927424002666>.
 - [10] K. JIANG, X. SU, AND J. ZHANG, *A general alternating-direction implicit framework with gaussian process regression parameter prediction for large sparse linear systems*, SIAM Journal on Scientific Computing, 44 (2022), pp. A1960–A1988, <https://doi.org/10.1137/21M1450197>, <https://doi.org/10.1137/21M1450197>, <https://arxiv.org/abs/https://doi.org/10.1137/21M1450197>.
 - [11] C. B. MOLER, *Iterative refinement in floating point*, J. ACM, 14 (1967), p. 316–321, <https://doi.org/10.1145/321386.321394>, <https://doi.org/10.1145/321386.321394>.
 - [12] NVIDIA, *Hopper architecture whitepaper*, 2022, <https://resources.nvidia.com/en-us-tensor-core/gtc22-whitepaper-hopper>. Accessed: 2024-12-10.
 - [13] NVIDIA, *Nvidia tensor core gpu datasheet*, 2022, <https://resources.nvidia.com/en-us-tensor-core/nvidia-tensor-core-gpu-datasheet>. Accessed: 2024-12-10.
 - [14] G. W. STEWART, *Matrix algorithms*, Society for Industrial and Applied Mathematics, USA, 2001.
 - [15] J. H. WILKINSON AND J. H. ET AL., *Rounding errors in algebraic processes*, Society for Industrial and Applied Mathematics, Philadelphia, Pennsylvania, 2023.
 - [16] C. WILLIAMS AND C. RASMUSSEN, *Gaussian processes for regression*, in Advances in Neural Information Processing Systems, D. Touretzky, M. Mozer, and M. Hasselmo, eds., vol. 8, MIT Press, 1995, https://proceedings.neurips.cc/paper_files/paper/1995/file/7cce53cf90577442771720a370c3c723-Paper.pdf.